

# Rapport Big Data

par Adefolami Christana, Launay Louise et Makosso Anne-Elisabeth

Liens github : <https://github.com/christana22> , <https://github.com/louise-33> & <https://github.com/AnneliMks>

Lien github projet : <https://github.com/AnneliMks/projet-big-data>

## I . Introduction

Le but de notre projet est d'analyser le marché immobilier de la vente dans le département du Rhône. Le choix du département du Rhône comme zone d'exploration s'explique par deux raisons :

- c'est un marché assez large pour nous permettre de porter une analyse complète et robuste mais tout de même limitée
- c'est un marché plutôt homogène, (contrairement au marché de la région parisienne qui est très complexe).

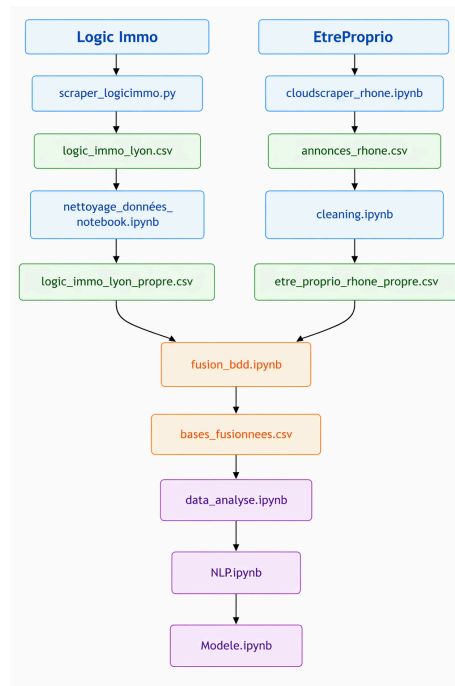
Nous avons extrait nos données de deux sites immobiliers : EtreProprio et Logic-Immo. Même si, beaucoup de plateformes immobilières sont complexes à scraper en raison des mesures de sécurité (anti-bot). En choisissant ces deux sites, nous avons été en mesure d'extraire respectivement près de 600 et 9 000 annonces

## II. Méthodologie et Architecture

Les données web scrappées ont d'abord été récupérées au format json, puis nous avons extrait les caractéristiques qui nous intéressaient pour les stocker dans des Data Frames, que nous avons ensuite exportées au format csv.

Nous avons travaillé de manière collaborative sur GitHub avec quatre branches : la branche main et trois branches correspondant à "l'espace de travail" de chaque membre du groupe.

Les données ont été récupérées, nettoyées, jointes et ensuite analysées et mises dans les modèles avec plusieurs notebooks, en suivant la structure ci-dessous.



### III. Acquisition et Préparation des Données

Comme dit précédemment nous avons récupéré les données au travers de deux sources. Globalement, les bases de données étaient plutôt propres mais il a fallu faire un peu de feature engineering sur les colonnes des bases de données pour avoir une certaine cohérence et pouvoir fusionner les deux bases de données par la suite.

#### Bases de données Logic Immo

La première base de données a été récupérée sur le site Logic Immo grâce à l'utilisation de la librairie Selenium. Bien que cette librairie puisse être plus lente que d'autres méthodes, elle est très intuitive et permet de simuler un comportement humain (clics, temps d'attente), ce qui est efficace pour contourner certaines sécurités. En amont, il a fallu faire des repérages sur la page de Logic Immo afin de comprendre la structure HTML du site internet et savoir quelles caractéristiques il fallait récupérer. Pour des questions de facilité et de rapidité, il est préférable de web scraper les pages avec toutes les annonces sans réaliser des clics sur chacune des annonces ce qui nous permet d'avoir accès aux caractéristiques suivants pour la plupart des annonces : prix (de vente), type de bien (maison, appartement), détails du bien (nombre de pièces, nombre de chambres, surface du bien, surface du terrain, étages).

Près de 9 000 annonces ont pu être récupérées sur ce site. Pour permettre l'utilisation de ces données pour le modèle et obtenir une base de données propre nous avons effectué des vérifications et des traitements au travers de plusieurs étapes :

- **Nettoyage global initial** : Suppression des doublons et des lignes où le nombre de pièces et de chambres était simultanément manquant.

- **Extraction grâce aux expressions régulières (Regex)** : Plusieurs caractéristiques étaient dans une seule colonne. Nous avons utilisé des Regex et des mots-clés (chambres, pièce, étages, terrain, m<sup>2</sup>) pour créer les bonnes colonnes. Pour la colonne ville, une liste des communes du Rhône a été utilisée.
- **Traitement des valeurs manquantes** : Nous avons appliqué des règles logiques. Par exemple, nous avons considéré que pièces = chambres + 1 dans les cas où le nombre de pièces est dans la médiane, au-delà nous avons déduit grâce à des techniques de regroupement. Pour les cas plus complexes des variables continues, nous avons utilisé la médiane calculée par groupes (ex : médiane de la surface par type de bien et nombre de pièces) pour imputer les données manquantes de manière réaliste.
- **Traitement des valeurs extrêmes** : Nous avons utilisé la technique de la winsorisation au 99ème percentile pour limiter l'impact des valeurs extrêmes dans le cas des variables continues. Pour la variable type de biens, un regroupement a été effectué pour n'avoir que les valeurs "maison" ou "appartement".

Une fois cela effectué, il fallait bien évidemment vérifier la cohérence des données : prix cohérent, nombre de chambres inférieur au nombre de pièces, appartement sans terrain et maison sans étages (correspond à l'étage où se situe l'appartement).

### Bases de données EtreProprio

La deuxième base de données a été récupérée à partir du site EtreProprio. Cette extraction a utilisé la library Cloudscraper, conçue pour éviter les protections contre les bots, comme Cloudflare notamment, et qui est plus rapide que Selenium. Nous avons également analysé la structure html du site, afin de scraper les informations qui nous intéressaient: le titre, une courte description, le prix, la description complète (souvent très longue), le DPE, et le nombre de pièces, ainsi que le lien précis de chaque annonce. Toujours pendant le web scraping, nous avons également extrait le type de bien à partir du titre, et le numéro de l'étage à partir de la description complète, à l'aide d'expressions régulières. Il a été ainsi possible de récupérer 580 annonces, un nombre plus bas que sur Logic Immo car le site EtreProprio ne rend accessible que les 30 premières pages de chaque recherche. La surface n'a pas toujours été correctement récupérée, donc elle a été reprise dans ces cas à partir du titre, où elle se trouve presque toujours, et de la description sinon.

Nous avons ensuite utilisé Pyspark pour un nettoyage des données plus complet. La base ne contenant que 580 lignes, nous avons choisi de ne pas supprimer de lignes sauf si cela était vraiment inévitable, et d'imputer les valeurs manquantes autant que possible.

- Lorsque le DPE était manquant, nous avons si possible essayé de le trouver dans la description complète. Une valeur a également été rajoutée pour les biens où le DPE était indiqué comme non applicable, ainsi que pour les biens où il est simplement inconnu, pour quelque raison.
- En l'absence de description complète, elle a simplement été remplacée par la description courte.
- Afin de retrouver le nombre manquant de pièces, le choix a été fait de calculer le ratio surface/pièces, en prenant la médiane par intervalle de surface et par type de bien (appartement ou maison). En effet, le nombre de pièces est directement dépendant

de la surface, et la taille des pièces varie souvent selon ces facteurs. Le ratio correspondant a donc été appliqué.

- Pour l'étage, nous avons codé les biens où la description indique simplement "dernier étage" par exemple, en tant que -1, car ils donnent tout de même une information sur l'étage même si l'étage lui-même n'est pas indiqué. Pour les biens où aucune information du tout sur l'étage n'est donnée, nous avons regardé la médiane par intervalle de surface pour les appartements afin de tenter de voir si une relation était remarquée, mais l'étage médian était le 1 dans chaque cas, donc c'est ce que nous avons mis. Les maisons ont simplement été codées en 0, car cette variable ne leur est pas pertinente.
- En analysant les valeurs extrêmes, il est devenu apparent avec la description que les deux biens qui ressortaient étaient en réalité des immeubles complets mis en vente. Cela n'étant pas pertinent dans le contexte voulu, et n'apportant donc aucune information importante, ces biens ont donc été supprimés.

Après traitement, la base extraite se compose de 578 lignes.

### Fusion des deux bases de données

Il a ensuite fallu regrouper les deux bases de données. Pour harmoniser les données, nous avons traité la colonne ville, afin de normaliser les noms des villes et renommer les villes dont l'ancien nom est utilisé, pour avoir les noms à jour (par exemple, Oullins et Pierre-Bénite ont été fusionnées en Oullins-Pierre-Bénite). Dans la base EtreProprio, la colonne étage est numérique (int), avec les appartements au rez-de-chaussé et les maisons tous deux codés 0, et nous avons donc transformé ces valeurs en RDC et MAISON en passant la colonne en string, pour correspondre à l'autre base. La variable correspondant au code postal a également été créée, à l'aide d'une liste de toutes les communes du Rhône et le code postale correspondant. Au final, les bases ont 7 colonnes en commun, que nous avons gardées dans la base fusionnée: type\_bien, pieces, prix, surface, etage, ville et code\_postal. Nous avons ensuite analysé et supprimé les doublons sur ces colonnes, pour ne garder que les offres uniques, et ne pas doubler celles qui auraient été postées sur les deux sites séparément.

Après traitement, la base jointe se compose de 8019 lignes.

## IV. Analyse Descriptive et Statistiques

Ce projet sur les données immobilières nous a permis d'analyser le marché de la vente de l'immobilier dans le département du Rhône. Nous avons ainsi pu remarquer plusieurs caractéristiques liées à ce marché :

- Il y a davantage d'appartements sur le marché de l'immobilier de ventes du Rhône que de maisons. En effet, parmi toutes les annonces récoltées, les appartements représentent 68,5% des annonces.
- On observe une asymétrie à droite dans la distribution des prix à cause des valeurs extrêmes : on se retrouve ainsi avec une médiane à 349 000 € et une moyenne à 427 315,2 €. On remarque que Lyon ne fait pas partie du top 10 des villes les plus chères

du Rhône en moyenne mais se trouve au contraire proche de la moyenne. Ce sont les petites villes huppées du Rhône qui ont les prix moyens les plus élevés (jusqu'à 2 millions d'euros). Les maisons les plus chères se trouvent pour la plupart en périphérie de Lyon mais les appartements en vente les plus chers se trouvent bien à l'intérieur de Lyon même. Lorsque l'on regarde le prix au m<sup>2</sup>, Lyon se trouve en 9ème position mais est très proche du top 1.

## V. Modélisation Prédictive

### Analyse NLP des descriptions : Procédure MapReduce

Afin d'exploiter le contenu textuel des annonces immobilières collectées sur EtreProprio, nous avons mis en œuvre une procédure de type MapReduce, telle qu'étudiée en cours, pour extraire les mots les plus fréquents des 578 descriptions d'annonces. Avant d'appliquer le MapReduce, chaque description a été nettoyée selon les étapes suivantes :

- mise en minuscules et suppression des caractères non alphabétiques par expression régulière
- suppression des mots de moins de 4 caractères
- filtrage d'une liste de stopwords français et anglais (articles, prépositions, pronoms)

La procédure MapReduce a été implémentée manuellement en Python, conformément à l'approche vue en cours avec les fonctions `groupByKey` et `reduceByKey`. Les 578 descriptions ont produit 74 926 paires (mot, 1) après l'étape Map, soit la mise en correspondance de chaque mot avec l'occurrence 1. L'étape Reduce a ensuite agrégé ces paires par clé, aboutissant à 5 275 mots distincts. Le classement des 5 mots les plus fréquents dans les descriptions est le suivant : appartement (797), salle (725), espace (609), lyon (595), cuisine (549) chambres (501), maison (491), copropriété (474), étage (439), résidence (388). Ces résultats sont cohérents avec le marché immobilier analysé. Les termes dominants (appartement, salle, cuisine, chambres) reflètent les pièces principales décrites dans les annonces. La forte présence de "lyon" confirme la concentration géographique du marché sur la métropole lyonnaise.

### Modélisation prédictive du prix de vente

#### Présentation des données et variables

La modélisation a été réalisée sur la base fusionnée de 8 019 annonces du département du Rhône. La variable à prédire est le prix de vente (en euros). Les variables explicatives retenues sont :

- `type_bien` : appartement ou maison (encodé par `StringIndexer`)
- `pieces` : nombre de pièces (variable numérique continue)
- `surface` : surface en m<sup>2</sup> (variable numérique continue)
- `etage` : étage ou situation du bien (MAISON, RDC, 1 à 18, encodé par `StringIndexer`)
- `ville_idx` : commune du Rhône (encodée par `StringIndexer` parmi 200+ modalités)
- `code postal` : code postal (variable numérique, localisation fine dans le Rhône)

L'encodage des variables catégorielles a été réalisé avec StringIndexer de PySpark ML. L'ensemble des features a été rassemblé dans un VectorAssembler. Le jeu de données a été découpé en un échantillon d'apprentissage (80%, soit 6 486 lignes) et un échantillon de test (20%, soit 1 533 lignes), avec une graine fixée à 42 pour la reproductibilité.

### Modèles

Trois modèles de régression ont été entraînés via PySpark MLlib, chacun dans un Pipeline et optimisé par validation croisée : Arbre de décision, random forest et xgboost. Les performances des trois modèles sur l'échantillon de test sont présentées dans le tableau ci-dessous, évaluées selon trois métriques : le coefficient de détermination  $R^2$ , la RMSE (Root Mean Square Error) et la MAE (Mean Absolute Error). **Le Random Forest est le meilleur modèle avec un  $R^2$  de 0.6957, une RMSE de 169 602 € et une MAE de 97 815 €.** Il explique ainsi près de 70% de la variance des prix de vente dans le Rhône. La Forêt Aléatoire surpasse le GBT et l'Arbre de Décision, ce qui est attendu : en tant que méthode d'ensemble basée sur le bagging, elle réduit la variance et est moins sensible aux valeurs extrêmes que l'Arbre de Décision seul. Afin d'identifier les variables les plus prédictives, nous avons extrait les feature importances des deux meilleurs modèles (Random Forest et GBT) :

| Variable (random forest) | Importance | Variable GBT | Variable |
|--------------------------|------------|--------------|----------|
| Surface                  | 0.441      | Surface      | 0.485    |
| ville_idx                | 0.227      | ville_idx    | 0.266    |
| pièces                   | 0.207      | etage_idx    | 0.105    |
| etage_idx                | 0.053      | pièces       | 0.080    |
| code postale             | 0.053      | code postale | 0.062    |
| type_idx                 | 0.015      | type_idx     | 0.001    |

Les deux modèles s'accordent sur un classement cohérent avec la logique du marché immobilier. La surface est la variable la plus prédictive (44% d'importance pour le RF, 49% pour le GBT), ce qui est logique à la réalité : le prix d'un bien est proportionnel à sa surface habitable. La localisation (ville\_idx et code\_postal combinés) représente environ 28% de l'importance totale pour le RF, confirme ainsi l'hétérogénéité des prix entre les communes du Rhône un appartement de même surface ne se vend pas au même prix à Lyon 6e qu'à Vaulx-en-Velin. Le nombre de pièces est également significatif (21% pour le RF), cela soit fortement corrélé à la surface. Le type de bien (appartement/maison) contribue peu (1.5%), son effet étant capturé en grande partie par les autres variables

### Analyse des résidus

L'analyse des résidus du modèle Random Forest révèle deux points importants. D'une part, la distribution des résidus est centrée autour de zéro et suit approximativement une loi normale, ce qui indique l'absence de biais systématique sur la majorité des biens. D'autre

part, le graphique prix réel vs prix prédit montre que le modèle est précis sur le segment 100 000 € - 1 000 000 €, qui représente l'essentiel du marché dans le Rhône. En revanche, le modèle sous-estime systématiquement les biens au-delà de 1 000 000 €. Ce comportement s'explique par la sous-représentation de ces biens dans les données scrapées (quelques dizaines d'annonces sur 8 019), qui ne permettent pas à l'algorithme d'apprendre correctement leurs spécificités.

## VI. Analyse Critique et Utilisation de l'IA

Étant donné les délais de rendu et notre manque d'expérience en Spark et en web scraping, l'utilisation de l'IA générative a été d'une grande aide. En effet, même s'il est évident que la vérification humaine est nécessaire, l'utilisation de l'IA générative a fortement facilité notre projet. Malgré l'utilisation de l'IA, on se rend compte qu'à force de l'utiliser, de demander des explications et de voir certaines commandes on commence d'une part à comprendre la structure globale du code généré mais également à pouvoir générer nous-mêmes de petites parties de code basiques utiles pour les vérifications.

En résumé, même si l'IA a bien évidemment été indispensable sur ce projet, il est important de pouvoir remettre en question certaines parties du code généré et de vérifier les résultats pour mener à bien ce projet. Cela passe aussi par l'utilisation de plusieurs IA pour parfois pouvoir débloquer certains problèmes.

## VII. Conclusion

### **Bilan technique et humain**

Ce projet nous a permis de réaliser un projet de data science de bout en bout, depuis la collecte des données jusqu'à la prédiction des prix. La partie la plus difficile a été le web scraping, car les sites immobiliers ont des protections anti-bot qui nous ont obligés à choisir des bibliothèques adaptées. Le nettoyage et le traitement des données avec PySpark ont aussi demandé beaucoup de travail, surtout pour harmoniser deux bases de données qui ne venaient pas du même site.

Le travail en équipe sur GitHub avec une branche par personne nous a permis d'avancer chacun de notre côté sans bloquer les autres. C'est une méthode de travail que nous n'avions pas vraiment pratiquée avant et qui s'est révélée très utile.

### **Perspectives**

Ce projet pourrait être amélioré de plusieurs façons. Il serait intéressant d'utiliser des données de transactions immobilières réelles, comme les DVF disponibles en open data, plutôt que des prix d'annonces qui ne correspondent pas toujours aux prix de vente finaux. On pourrait aussi essayer d'intégrer les descriptions textuelles des annonces comme variables dans le modèle, ce qui permettrait de capturer des informations comme la présence d'un parking ou d'une terrasse. Enfin, déployer le modèle sous forme d'application web permettrait à n'importe quel utilisateur d'estimer le prix d'un bien dans le Rhône en quelques clics.

